Class and Objects

```
# base class / parent class
class Phone:
  category = "Electronics"

  # constructor
  def __init__(self,model , battery , camera,battery_percentage=100):
    self.model = model
    self.battery = battery
    self.camera = camera
    self.battery_percentage = battery_percentage

  # methods
  def charge(self,hour):
    print(f"charge completed by {hour} {model}")

  def capture(self,photo):
    if(self.battery_percentage)<=0:
      print("No charge")
    else:
      self.battery_percentage-=photo
      print(f"photo captured in {self.model}")



class Cooling_Mechanism:
  def __init__(self,cooling_method):
    self.cooling_method= cooling_method


  #methods
  def cooling_on(self):
    print(f"the system is being cool by {self.cooling_method}")
```

```
## creating object

apple = Phone("Iphone17",3000,40)
blueberry = Phone("B-17",4000,30)
motorola = Phone("M-17",3500,35)
apple.capture(10)
print(motorola.battery_percentage)
```

```
photo captured in Iphone17
100
```

Single Inheritance

```
# child / derived class
class SmartPhone(Phone):
```

```
  def __init__(self,model , battery , camera,processor):
    super().__init__(model,battery,camera)
    self.processor = processor

   ## modified method
  def charge(self,hour):
    print("fast charging in process")
    super().charge(hour)
```

```
pro = SmartPhone("X",5000,100,"SnapDragon")

print(pro.model)

pro.charge(1)
```

```
X
fast charging in process
charge completed by 1
```

```
Start coding or generate with AI.
```

Multiple Inheritance

```
class SmartPhone_CoolingMode(SmartPhone,Cooling_Mechanism):
   def __init__(self,model , battery , camera,processor,cooling_method):
    SmartPhone.__init__(self,model,battery,camera,processor)
    Cooling_Mechanism.__init__(self,cooling_method)
```

```
pro_cooling = SmartPhone_CoolingMode("Y",5000,100,"SD","Nitrogen")

print(pro_cooling.processor) # SmartPhone class theke
print(pro_cooling.battery) # Phone class theke
print(pro_cooling.cooling_method) # Cooling_Mechanism class theke
print(pro_cooling.cooling_on()) # Cooling_Mechanism class er method
print(pro_cooling.charge(1)) # SmartPhone er modified charge jeta Phone class theke inherited
```

```
SD
5000
Nitrogen
the system is being cool by Nitrogen
None
fast charging in process
charge completed by 1
None
```

```
Start coding or generate with AI.
```

Polymorphsim

```
```

```
class Camera:
  def __init__(self , name):
    self.name = name

 # method
  def capture(self):
    print("a photo is captured")
```

```
class Smart_Phone(Camera):
  def __init__(self,name,resolution):
    super().__init__(name)
    self.resolution = resolution

  #method overriding
  def capture(self):
    print("Photo is captured by a Phone")


class DSLR(Camera):
  def __init__(self,name,resolution):
    super().__init__(name)
    self.resolution = resolution

  def capture(self):
    print("Photo is captured  by DSLR")



class Drone(Camera):
  def __init__(self,name,resolution):
    super().__init__(name)
    self.resolution = resolution

  def capture(self):
    print("Photo is captured by Drone")



phone = Smart_Phone("Phone",30)
dslr = DSLR ("DSLR",200)
drone = Drone("Drone",150)


phone.capture()
dslr.capture()
drone.capture()
```

```
Photo is captured by a Phone
Photo is captured  by DSLR
Photo is captured by Drone
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

Encapsulation

```
```

```
# design
class Mobile:
  def __init__(self,name,model,imei):
    self.__name = name
    self.__model = model
    self.__imei = imei # private

  def charge(self):
    print("phone is charging")

  # method imei ta paite pari
  def imei_getter(self):
    return self.__imei




  def name_getter(self):
    return self.__name


  def name_setter(self,name):
    self.__name = name







#outside world ( user )

# registration
iphone = Mobile("Phone","17","1xkaf1")

print(iphone.name_getter())

iphone.name_setter("Phitron")


print(iphone.name_getter())
```

```
Phone
Phitron
```

```
Start coding or generate with AI.
```

Abstraction

```
from abc import  ABC , abstractmethod

class Telephone(ABC):

  @abstractmethod
  def make_call(self):
    pass
```

```
class Sphone(Telephone):
  def make_call(self):
    print("Making a call using SPhone")



class Iphone(Telephone):
   def make_call(self):
    print("Making a call using IPhone")
```

```
ip = Iphone()

ip.make_call()
```

```
Making a call using IPhone
```

```
Start coding or generate with AI.
```